# ESTRUCTURA DE DATOS

## Guía Práctica Hands-on — Semana 1

*Listas Enlazadas Simples en C# | Aplicación de Consola*
*Docente: Dr. Eric Gustavo Coronel Castillo*

---

## 1. Objetivos de Aprendizaje

Al finalizar esta guía, el estudiante será capaz de:

- Comprender la estructura interna de un nodo en una lista enlazada simple.
- Implementar las operaciones de creación, inserción y recorrido de una lista enlazada simple en C#.
- Desarrollar una solución funcional en una Aplicación de Consola usando buenas prácticas de programación.

## 2. Contexto del Problema

Una tienda de conveniencia necesita registrar los precios de sus productos de forma dinámica, sin conocer de antemano cuántos artículos ingresará. Para ello, se implementará una Lista Enlazada Simple que permita:

- Crear la lista vacía.
- Agregar productos (nombre y precio) al final de la lista.
- Listar todos los productos registrados.

*Este escenario es ideal para aplicar listas enlazadas porque la cantidad de productos varía en tiempo de ejecución y no se requiere acceso aleatorio por índice.*

## 3. Conceptos Clave Antes de Comenzar

### 3.1 ¿Qué es un Nodo?

Un nodo es la unidad básica de una lista enlazada. Contiene dos partes:

- Dato: el valor almacenado (en este caso, nombre y precio del producto).
- Enlace (Siguiente): una referencia al siguiente nodo de la lista. Si es el último nodo, este enlace apunta a null.

### 3.2 ¿Qué es una Lista Enlazada Simple?

Es una secuencia de nodos donde cada nodo conoce únicamente al nodo siguiente. El recorrido solo puede hacerse en una dirección: de la cabeza (head) hacia el final (null).

**Representación gráfica:**

```
[ Coca-Cola / S/2.50 | → ] → [ Leche / S/3.80 | → ] → [ Pan / S/1.00 | null ]
```

*Head apunta al primer nodo. El último nodo tiene Siguiente = null.*

## 4. Configuración del Proyecto en Visual Studio

### Paso 1: Crear el Proyecto

1. Abrir Visual Studio (2022 o superior).
2. Seleccionar Crear un nuevo proyecto.
3. Elegir la plantilla: Aplicación de consola (.NET) con lenguaje C#.
4. Asignar el nombre: ListaSimpleProductos.
5. Seleccionar .NET 6.0 o superior y hacer clic en Crear.

### Paso 2: Estructura de Archivos

El proyecto contendrá los siguientes archivos que crearemos en los pasos siguientes:

- Nodo.cs — Define la clase Nodo.
- ListaSimple.cs — Define la clase ListaSimple con sus operaciones.
- Program.cs — Punto de entrada de la aplicación.

# 5. Implementación Paso a Paso

## Paso 5.1: Crear la Clase Nodo

En el Explorador de Soluciones, hacer clic derecho sobre el proyecto → Agregar → Clase. Nombrarla Nodo.cs.

Escribir el siguiente código en el archivo Nodo.cs:

```
// Clase que representa un nodo de la lista enlazada
public class Nodo
{
  // Datos del producto
  public string Nombre { get; set; }
  public double Precio { get; set; }

  // Enlace al siguiente nodo
  public Nodo Siguiente { get; set; }

  // Constructor: recibe nombre y precio
  public Nodo(string nombre, double precio)
  {
    Nombre    = nombre;
    Precio    = precio;
    Siguiente = null;  // Por defecto apunta a null
  }
}
```

**Nota conceptual:**
La propiedad Siguiente es de tipo Nodo, lo que permite que cada nodo "conozca" al siguiente de la misma clase. Cuando vale null, indica el fin de la lista.

### Paso 5.2: Crear la Clase ListaSimple

Crear un nuevo archivo llamado ListaSimple.cs y escribir el siguiente código:

```
public class ListaSimple
{
  // Referencia al primer nodo (cabeza de la lista)
  private Nodo cabeza;

  // Constructor: inicializa la lista vacía
  public ListaSimple()
  {
    cabeza = null;
  }

  // ──────────────────────────────────────────────
  // OPERACIÓN 1: Agregar un producto al final
  // ──────────────────────────────────────────────
  public void Agregar(string nombre, double precio)
  {
    Nodo nuevo = new Nodo(nombre, precio);

    if (cabeza == null)
    {
      // La lista está vacía: el nuevo nodo es la cabeza
      cabeza = nuevo;
    }
    else
    {
      // Recorrer hasta el último nodo
      Nodo actual = cabeza;
      while (actual.Siguiente != null)
      {
        actual = actual.Siguiente;
      }
      // Enlazar el nuevo nodo al final
      actual.Siguiente = nuevo;
    }
  }

  // ──────────────────────────────────────────────
  // OPERACIÓN 2: Listar todos los productos
  // ──────────────────────────────────────────────
  public void Listar()
  {
    if (cabeza == null)
```

```
    {
      Console.WriteLine("  La lista está vacía.");
      return;
    }

    Console.WriteLine("  {0,-25} {1,10}", "PRODUCTO", "PRECIO");
    Console.WriteLine("  " + new string('-', 38));

    Nodo actual = cabeza;
    int contador = 1;

    while (actual != null)
    {
      Console.WriteLine("  {0}. {1,-22} S/{2,7:F2}",
                    contador, actual.Nombre, actual.Precio);
      actual = actual.Siguiente;
      contador++;
    }
  }
}
```

**Puntos clave del método Agregar:**

- Se crea un nuevo nodo con los datos recibidos.
- Si la lista está vacía (cabeza == null), el nuevo nodo se convierte en la cabeza.
- Si la lista tiene elementos, se recorre hasta el último (actual.Siguiente == null) y se enlaza el nuevo nodo allí.

### Paso 5.3: Escribir el Programa Principal (Program.cs)

Abrir el archivo Program.cs y reemplazar su contenido con el siguiente código:

```
using ListaSimpleProductos;

Console.WriteLine("========================================");
Console.WriteLine("   GESTIÓN DE PRECIOS - TIENDA EXPRESS  ");
Console.WriteLine("========================================");

// 1. Crear la lista vacía
ListaSimple lista = new ListaSimple();

// 2. Agregar productos a la lista
lista.Agregar("Coca-Cola 600ml",   2.50);
lista.Agregar("Leche Gloria 1L",   3.80);
lista.Agregar("Pan de molde",      5.20);
lista.Agregar("Aceite vegetal 1L", 8.90);
lista.Agregar("Azúcar 1Kg",        4.30);

// 3. Mostrar la lista
Console.WriteLine();
Console.WriteLine("  Productos registrados:");
lista.Listar();

Console.WriteLine();
Console.WriteLine("========================================");
Console.ReadKey();
```

## 6. Resultado Esperado en Consola

Al ejecutar el proyecto (F5), la consola debe mostrar una salida similar a la siguiente:

```
========================================
   GESTIÓN DE PRECIOS - TIENDA EXPRESS
========================================

  Productos registrados:
  PRODUCTO                      PRECIO
  --------------------------------------
  1. Coca-Cola 600ml        S/   2.50
  2. Leche Gloria 1L        S/   3.80
  3. Pan de molde           S/   5.20
  4. Aceite vegetal 1L      S/   8.90
  5. Azúcar 1Kg             S/   4.30

========================================
```

✓ **Verificación:**

Si la salida coincide con lo mostrado arriba, la implementación de las operaciones Crear, Agregar y Listar es correcta. El recorrido de la lista mediante el puntero actual y el avance actual = actual.Siguiente es el mecanismo central que se debe comprender.

## 7. Análisis del Flujo de Ejecución

Comprender el flujo interno es fundamental. A continuación, se describe lo que ocurre en memoria al ejecutar el programa:

### 7.1 Al crear la lista

Se ejecuta el constructor de ListaSimple: cabeza = null. La lista existe, pero no contiene ningún nodo.

### 7.2 Al agregar el primer producto (Coca-Cola)

Se crea el nodo Nodo_1 con Nombre="Coca-Cola 600ml", Precio=2.50, Siguiente=null. Como cabeza==null, se asigna cabeza = Nodo_1.

### 7.3 Al agregar el segundo producto (Leche Gloria)

Se crea Nodo_2. Como cabeza != null, se recorre la lista. Se encuentra que Nodo_1.Siguiente == null, entonces Nodo_1.Siguiente = Nodo_2. La lista queda: cabeza → Nodo_1 → Nodo_2 → null.

### 7.4 Al listar

actual inicia en cabeza (Nodo_1). En cada iteración se imprime actual.Nombre y actual.Precio, luego actual = actual.Siguiente. El ciclo termina cuando actual == null.

## 8. Retos

Aplica lo aprendido extendiendo la solución propuesta. Cada reto debe resolverse sobre el mismo proyecto.

### Reto 1 — Insertar al Inicio de la Lista

Actualmente el método Agregar() inserta los productos al final. Implementa un nuevo método llamado AgregarAlInicio(string nombre, double precio) que inserte el nodo al principio de la lista, de modo que el nuevo producto quede como la nueva cabeza.

**Criterio de éxito:**

Si agregas "Galletas Oreo / S/3.50" al inicio antes de listar, debe aparecer como primer elemento en la consola.

***Pista conceptual:***

*El nuevo nodo debe apuntar a la cabeza actual antes de que cabeza sea reasignada.*

### Reto 2 — Contar Elementos de la Lista

Implementa un método llamado Contar() que recorra la lista y retorne el número total de productos registrados (tipo int).

**Criterio de éxito:**

En Program.cs, después de agregar los 5 productos, muestra en consola un mensaje como: "Total de productos: 5".

***Pista conceptual:***

*Usa un contador local que se incrementa en cada iteración del recorrido de la lista.*

## Reto 3 — Mostrar el Producto Más Caro

Implementa un método llamado ObtenerMasCaro() que recorra la lista y retorne el nodo (tipo Nodo) con el precio más alto. Si la lista está vacía, debe retornar null.

**Criterio de éxito:**

En Program.cs, después de listar todos los productos, muestra: "Producto más caro: Aceite vegetal 1L — S/8.90".

***Pista conceptual:***

*Inicializa una variable auxiliar con el primer nodo y actualízala si encuentras un nodo con mayor precio durante el recorrido.*

## 9. Preguntas de Comprensión

Selecciona la alternativa correcta para cada pregunta. Las preguntas evalúan la comprensión conceptual y la aplicación práctica de los contenidos de esta guía.

**Pregunta 1:** En la implementación desarrollada, ¿qué ocurre internamente cuando se invoca lista.Agregar("Leche", 3.80) sobre una lista que ya contiene un elemento?

A. Se sobrescribe el valor del nodo cabeza con los nuevos datos.
B. Se crea un nuevo nodo y se enlaza directamente a la cabeza sin recorrer la lista.
C. Se recorre la lista hasta encontrar el nodo cuyo campo Siguiente es null, y allí se enlaza el nuevo nodo.
D. Se lanza una excepción porque la lista solo admite un elemento.

**Pregunta 2:** ¿Cuál es la condición que permite al ciclo while del método Listar() saber que ha llegado al último nodo de la lista?

A. actual.Nombre == null
B. actual == cabeza
C. actual.Siguiente == cabeza
D. actual.Siguiente == null

**Pregunta 3:** Si se desea insertar un nuevo nodo al inicio de la lista (antes de la cabeza actual), ¿cuál es el orden correcto de las instrucciones?

A. cabeza = nuevo; nuevo.Siguiente = cabeza;
B. nuevo.Siguiente = cabeza; cabeza = nuevo;
C. nuevo.Siguiente = null; cabeza = nuevo;
D. cabeza = null; cabeza = nuevo;

## 10. Conclusiones

- Una lista enlazada simple crece dinámicamente; no es necesario declarar su tamaño de antemano, a diferencia de un arreglo.
- Cada nodo solo conoce al siguiente, lo que implica que el recorrido siempre es unidireccional (de cabeza hacia null).
- Las operaciones de inserción al final requieren recorrer la lista completa (O(n)); insertar al inicio es inmediato (O(1)).
- Comprender el manejo de referencias (punteros) es la habilidad fundamental para implementar cualquier variante de listas enlazadas: dobles, circulares, pilas y colas.